Adam Edwards
CS 414 Programming Language
Dr. Lewis

Assignment 07

Part 1 - C++ Nonvirtual Diamond Outputs

```
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp> cmake --build build --target nonvirtual_diamond
MSBuild version 18.10.1-1.26427.6+3cd27c13e for .NET Framework

  1>Checking Build System
  Building Custom Rule C:/Users/Adam/source/repos/cs414.edwards/Assignment07/cpp/CMakeLists.txt
  nonvirtual_diamond.cpp
  nonvirtual_diamond.vcxproj -> C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp\build\Debug\nonvirtual_diamond.exe
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp> .\build\Debug\nonvirtual_diamond.exe
Constructing the TeachingAssistant:
Person constructor
Student constructor
Person constructor
Employee constructor
TeachingAssistant constructor

Before the name change:
student_person  address: 000000045E55F7D0  name: Alex Rivera
employee_person address: 000000045E55F820  name: Alex Rivera

After changing the name through student_person:
student_person  name: Alexandra Rivera
employee_person name: Alex Rivera
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp>
```

Figure 1

```
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp> cmake --build build --target nonvirtual_diamond
MSBuild version 18.10.1-1.26427.6+3cd27c13e for .NET Framework

  nonvirtual_diamond.cpp
C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp\nonvirtual_diamond.cpp(86,25): error C2385: ambiguous access of 'name' [C:\Users\Adam\sourc
e\repos\cs414.edwards\Assignment07\cpp\build\nonvirtual_diamond.vcxproj]
      C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp\nonvirtual_diamond.cpp(86,25):
      could be the 'name' in base 'Person'
      C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp\nonvirtual_diamond.cpp(86,25):
      or could be the 'name' in base 'Person'

PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp>
```

Figure 2

```
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp> cmake --build build --target nonvirtual_diamond
MSBuild version 18.10.1-1.26427.6+3cd27c13e for .NET Framework

  nonvirtual_diamond.cpp
C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp\nonvirtual_diamond.cpp(89,23): error C2594: 'initializing': ambiguous conversions from 'Tea
chingAssistant *' to 'Person *' [C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp\build\nonvirtual_diamond.vcxproj]
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp>
```

Figure 3

Student and Employee each inherit their own copy of Person, so the names can be different. The Person constructor runs twice because each copy needs to be initialized. The compiler rejects ta.name and Person* person = &ta because it cant tell which Person to use.

A class is the definition of an object, while a base-class subobject is the actual inherited part stored inside an object.

Part 2 - C++ Virtual Diamond Outputs

```
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp> cmake --build build --target nonvirtual_diamond
MSBuild version 18.10.1-1.26427.6+3cd27c13e for .NET Framework

  nonvirtual_diamond.cpp
  nonvirtual_diamond.vcxproj -> C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp\build\Debug\nonvirtual_diamond.exe
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp> cmake --build build --target virtual_diamond
MSBuild version 18.10.1-1.26427.6+3cd27c13e for .NET Framework

  Building Custom Rule C:/Users/Adam/source/repos/cs414.edwards/Assignment07/cpp/CMakeLists.txt
  virtual_diamond.cpp
  virtual_diamond.vcxproj -> C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp\build\Debug\virtual_diamond.exe
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp> .\build\Debug\virtual_diamond.exe
Constructing the TeachingAssistant:
Person constructor
Student constructor
Employee constructor
TeachingAssistant constructor

Before the name change:
student_person  address: 0000000D1F1EFAA0  name: Alex Rivera
employee_person address: 0000000D1F1EFAA0  name: Alex Rivera

After changing the name through student_person:
student_person  name: Alexandra Rivera
employee_person name: Alexandra Rivera

Expressions rejected in Part 1:
ta.name: Alexandra Rivera
Person* person = &ta  address: 0000000D1F1EFAA0  name: Alexandra Rivera

Virtual role() through Person&:
person_ref.role(): Teaching Assistant

Part 4 - announce_person:
Teaching Assistant

Part 4 - announce_generic:
Teaching Assistant
Robot Assistant
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp>
```

Figure 4

```
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp> cmake --build build --target virtual_diamond
MSBuild version 18.10.1-1.26427.6+3cd27c13e for .NET Framework

  virtual_diamond.cpp
C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp\virtual_diamond.cpp(65,7): error C2250: 'TeachingAssistant': ambiguous inheritance of 'std:
:string Person::role(void) const' [C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp\build\virtual_diamond.vcxproj]
PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07\cpp>
```

Figure 5

Virtual inheritance fixes the duplicate Person problem by making Student and Employee share one copy. TeachingAssistant has to initialize Person because its the most-derived class, and Person doesnt have a default constructor.

Virtual inheritance doesnt fix competing overrides because Student and Employee both override role(). The compiler cant choose between them, so TeachingAssistant needs its own override.

Part 3 - Python Cooperative Diamond Outputs

```
(base) PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07> python python\cooperative_diamond.py
Constructing the TeachingAssistant:
Entering TeachingAssistant.__init__  id(self) = 2173607317520
Entering Student.__init__            id(self) = 2173607317520
Entering Employee.__init__           id(self) = 2173607317520
Entering Person.__init__             id(self) = 2173607317520
id(ta) after construction:           2173607317520

TeachingAssistant.__mro__:
(<class '__main__.TeachingAssistant'>, <class '__main__.Student'>, <class '__main__.Employee'>, <class '__main__.Person'>, <class 'object'>)

vars(ta):
{'course': 'CS 240', 'major': 'Computer Science', 'employee_id': 1042, 'name': 'Alex Rivera'}

Name changes:
After set_student_name:  ta.name = Alexandra Rivera
After set_employee_name: ta.name = A. Rivera

ta.role():
Teaching Assistant

Constructing the ReversedTeachingAssistant:
Entering ReversedTeachingAssistant.__init__  id(self) = 2173607317856
Entering Employee.__init__           id(self) = 2173607317856
Entering Student.__init__            id(self) = 2173607317856
Entering Person.__init__             id(self) = 2173607317856

ReversedTeachingAssistant.__mro__:
(<class '__main__.ReversedTeachingAssistant'>, <class '__main__.Employee'>, <class '__main__.Student'>, <class '__main__.Person'>, <class 'object'>)

reversed_ta.role():
Employee

Part 4 - announce():
Teaching Assistant
Robot Assistant
AttributeError: 'int' object has no attribute 'role'

Program finished normally.
(base) PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07>
```

Figure 6

Prediction: ta.role() should return "Student" because Python follows the MRO and checks Student before Employee. Since TeachingAssistant doesnt have its own role() method, Python should use the first one it finds.

```
(base) PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07> python python\cooperative_diamond.py
Constructing the TeachingAssistant:
Entering TeachingAssistant.__init__  id(self) = 1828916203536
Entering Student.__init__            id(self) = 1828916203536
Entering Employee.__init__           id(self) = 1828916203536
Entering Person.__init__             id(self) = 1828916203536
id(ta) after construction:           1828916203536

TeachingAssistant.__mro__:
(<class '__main__.TeachingAssistant'>, <class '__main__.Student'>, <class '__main__.Employee'>, <class '__main__.Person'>, <class 'object'>)

vars(ta):
{'course': 'CS 240', 'major': 'Computer Science', 'employee_id': 1042, 'name': 'Alex Rivera'}

Name changes:
After set_student_name:  ta.name = Alexandra Rivera
After set_employee_name: ta.name = A. Rivera

ta.role():
Student

Constructing the ReversedTeachingAssistant:
Entering ReversedTeachingAssistant.__init__  id(self) = 1828916203872
Entering Employee.__init__           id(self) = 1828916203872
Entering Student.__init__            id(self) = 1828916203872
Entering Person.__init__             id(self) = 1828916203872

ReversedTeachingAssistant.__mro__:
(<class '__main__.ReversedTeachingAssistant'>, <class '__main__.Employee'>, <class '__main__.Student'>, <class '__main__.Person'>, <class 'object'>)

reversed_ta.role():
Employee

Part 4 - announce():
Student
Robot Assistant
AttributeError: 'int' object has no attribute 'role'

Program finished normally.
(base) PS C:\Users\Adam\source\repos\cs414.edwards\Assignment07>
```

Figure 7

The initializers all show the same id(self) because Python uses one object instead of creating separate copies like C++. This is also why there is only one name attribute. Both Student and Employee change the same value, so the last change wins.

The MRO determines the order Python searches for methods. TeachingAssistant checks Student before Employee, so it uses Student.role() first. Reversing the base class order makes Employee come first instead.

super() follows the next class in the MRO, not necessarily the direct parent. This allows each initializer to run in order without calling the same class twice.

Part 4 - Polymorphism Without Common Ancestry Outputs

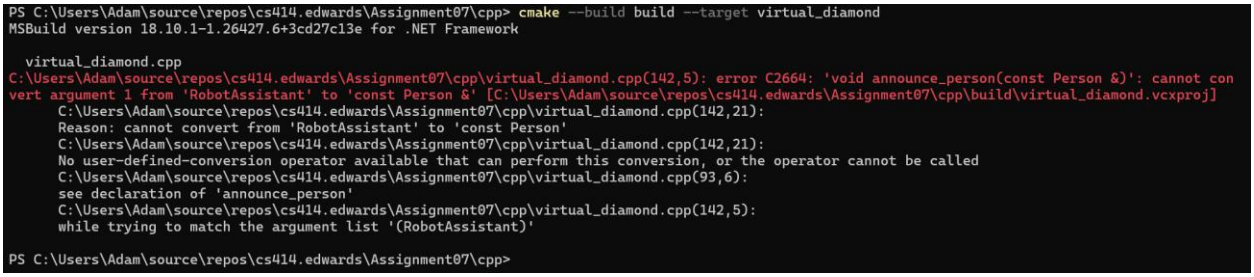


Figure 8

C++ inheritance-based polymorphism uses a Person reference and virtual functions to call the correct role() method at runtime. RobotAssistant doesnt inherit from Person, so the compiler rejects it even though it has a role() method.

C++ templates work differently because they dont require inheritance. They just need the object to have a role() method. Both TeachingAssistant and RobotAssistant work, but a missing method causes a compile-time error.

Python uses duck typing, which also doesnt require inheritance. It works as long as the object has a role() method. Unlike C++, Python doesnt catch a missing method until runtime, when it raises an AttributeError.

Part 5 - Analysis Report

1. What state is duplicated in the nonvirtual C++ diamond, and what changes under virtual inheritance?

In the nonvirtual version, Student and Employee each have their own copy of Person, including name. Virtual inheritance fixes this by making them share one Person, so changing the name through either side updates the same value.

2. How do virtual inheritance and virtual functions serve different purposes?

Virtual inheritance prevents duplicate copies of a base class. Virtual functions determine which version of a method runs at runtime. One handles shared data, while the other handles behavior.

3. Why does Pythons implementation avoid automatically creating two Person subobjects? Does that eliminate all possible initialization or naming conflicts?

Python uses one object instead of creating separate Person subobjects. The MRO makes sure Person only appears once in the initialization chain. Conflicts can still happen if classes use the same attribute names or dont properly call super().

4. Why can Python select an inherited role() using its MRO while the C++ virtual diamond requires a unique final override?

Python follows the MRO and uses the first role() method it finds. C++ doesnt automatically choose between Student and Employees overrides, so TeachingAssistant needs its own override to resolve the conflict.

5. Does sharing Pythons universal base object guarantee that a value supports role()? Explain using the integer experiment.

No. Just because everything inherits from object doesnt mean it has a role() method. When we passed 42 to announce(), Python raised an AttributeError at runtime because integers dont have that method.

6. Which design would you choose for this personnel model: multiple inheritance or composition with separate student and employee roles? Justify your choice.

I would choose composition because a person can have both student and employee roles without needing multiple inheritance. It would be easier to manage if someones roles change, and it avoids the duplicate data and method conflicts we saw in the experiments.